National University of Singapore
School of Computing
CS1101S: Programming Methodology
Semester I, 2026/2027

S12
Modifying the Metacircular Evaluator

Getting started

This studio modifies the metacircular evaluator of Lecture L11B. The evaluator handles numbers, names, the operators `+`, `-`, `*` and `/`, conditional expressions, sequences, and assignments. The function `parse_and_evaluate` parses a program and evaluates it in the global environment; the Repl tab of the Source Academy displays the value that it returns. Only `evaluate` and `apply` are shown here; the playground links on this sheet run the complete evaluator.

```
def evaluate(component, env):
    if is_literal(component):
        return literal_value(component)
    elif is_name(component):
        return lookup_symbol_value(
            symbol_of_name(component), env)
    elif is_operator_combination(component):
        return eval_operator_combination(component, env)
    elif is_conditional(component):
        return eval_conditional(component, env)
    elif is_sequence(component):
        return eval_sequence(
            sequence_statements(component), env)
    elif is_assignment(component):
        return eval_assignment(component, env)
    else:
        return error('unknown syntax', component)

def apply(operator, operands):
    first_op = head(operands)
    second_op = head(tail(operands))
    return (first_op + second_op if operator == '+' else
            first_op - second_op if operator == '-' else
            first_op * second_op if operator == '*' else
            first_op / second_op if operator == '/' else
            error('unknown operator -- apply', operator))
```

To see what the parser makes of a program, display its syntax tree. The following registers an evaluator that returns the syntax tree of the program for the Repl to display, here for `x = 4`:

```
set_evaluator(lambda program: parse(program))
set_program_text("x = 4")
```

To run a program, register `parse_and_evaluate` as the evaluator of the Repl tab. The program text of the Repl is prefilled; press Run in the Repl tab to evaluate it, or type your own programs. Names that a program assigns stay in the global environment for the programs that you run later:

```
set_evaluator(parse_and_evaluate)
set_program_text("x = 4\ny = x + 38\ny * 2")
```

Problems:

1. The language of our evaluator has no exponentiation operator. Modify the evaluator such that the operator `%` is the exponentiation operator, that is, `b % e` evaluates to b^e.

   Does `b % e % n` evaluate to b^{e^n}? Why? If not, what would need to change?

   The evaluator currently rejects the operator `%`. Try it in the Repl:

   ```
   set_evaluator(parse_and_evaluate)
   set_program_text("2 % 10")
   ```

2. Our evaluator does not detect unbound names. Therefore the following program runs without error in the evaluator, because the unbound name is never evaluated:

   ```
   set_evaluator(parse_and_evaluate)
   set_program_text("abracadabra if False else 42")
   ```

   The Source Academy, on the other hand, rejects this program before running it: *This name is not found in the current or enclosing environment(s).* Modify the evaluator such that any unbound name is detected and reported as an error before the program is evaluated. A name is bound if an earlier statement of the program assigns to it.

   Hint: walk through the program and check every name that it uses, keeping track of the names that have been assigned so far.

3. Our evaluator, like the Python of the Source Academy, only allows `True` and `False` as the value of the predicate of a conditional expression. Full Python allows *any* value there. The following values are "falsy": they lead to the evaluation of the alternative.

   - `False`
   - `0` and `0.0`
   - `""` (the empty string)
   - `None`

   All other values are "truthy": they lead to the evaluation of the consequent. Modify the evaluator such that it follows this design of full Python. For now, the evaluator reports an error for the following program:

```
set_evaluator(parse_and_evaluate)
set_program_text("1 if 0 else 2")
```

